Kaggle competition report.docx

AUT University

Document Details

Submission ID
trn:oid:::20437:155776354

Submission Date
Oct 25, 2026, 11:19 AM GMT+13

Download Date
Oct 25, 2026, 11:26 AM GMT+13

File Name
Kaggle competition report.txt

File Size
34.6 KB

16 Pages

5,764 Words

29,021 Characters

13% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

40 Not Cited or Quoted 13%
Matches with neither in-text citation nor quotation marks

1 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

11% Internet sources
9% Publications
11% Submitted works (Student Papers)

Match Groups

- **40** Not Cited or Quoted 13%
  Matches with neither in-text citation nor quotation marks
- **1** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

11% Internet sources
9% Publications
11% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
arxiv.org		3%
2	Submitted works	
Tilburg University on 2026-08-17		2%
3	Internet	
www.biorxiv.org		2%
4	Submitted works	
Thesis 2026 on 2026-08-06		<1%
5	Submitted works	
Sheffield Hallam University on 2026-09-10		<1%
6	Internet	
d197for5662m48.cloudfront.net		<1%
7	Submitted works	
Nottingham Trent University on 2026-05-08		<1%
8	Submitted works	
Kennesaw State University on 2023-02-03		<1%
9	Internet	
edoc.ub.uni-muenchen.de		<1%
10	Submitted works	
Helsingin Suomalainen Yhteiskoulu on 2026-08-21		<1%

11 Internet
ebin.pub <1%

12 Internet
tel.archives-ouvertes.fr <1%

13 Submitted works
Associatie K.U.Leuven on 2026-08-19 <1%

14 Submitted works
University at Buffalo on 2023-10-13 <1%

15 Internet
github.com <1%

16 Submitted works
United World College of the American West on 2026-10-21 <1%

17 Submitted works
University of East London on 2026-08-30 <1%

18 Internet
ora.ox.ac.uk <1%

19 Submitted works
University of East London on 2024-10-14 <1%

20 Internet
deepai.org <1%

21 Internet
ml4astro.github.io <1%

22 Submitted works
Associatie K.U.Leuven on 2026-08-26 <1%

23 Submitted works
City University on 2020-10-19 <1%

24 Submitted works
University of Technology, Sydney on 2020-06-23 <1%

25 Publication

Zoi A. Traga Philippakos, Eleni Katsarou, Konstantinos Sipitanos. "Artificial Intelli... <1%

26 Internet

aclanthology.org <1%

27 Internet

peerj.com <1%

28 Internet

raw.githubusercontent.com <1%

29 Publication

Paolo Ferro, Harinadh Vemanaboina, Chander Prakash. "Computational Techniqu... <1%

30 Internet

trepo.tuni.fi <1%

Task 1: Dataset Exploration
Dataset and classes
11
The images in the dataset are JPEG images. The dataset contains 63,660 labeled training images and 29,443 unlabeled test images. The images show one of the eight facial features of a cat or a dog. The eight views are CatEye, CatFace, CatMouth, CatNose, DogEye, DogFace, DogMouth, and DogNose. Every image in the labeled training set has a label, and every label corresponds to an image file. All the image files are usable.

Class balance
The classes are very imbalanced. The CatEye class has 13,884 examples, or 21.81% of all examples. The other three dog classes, DogFace, DogMouth, and DogNose, each have 5,790 examples, or 9.10% of all examples. The largest class has more than twice as many examples as the smallest class. A trivial model that always classifies an example as CatEye has an accuracy of 21.8% for the majority class. Stratified split is used in Section 2 to mitigate class imbalance, and per-class accuracies as well as overall accuracies are reported for all methods.

Class
Images
Share (%)
CatEye
13,884
21.81
CatFace
6,942
10.90
CatMouth
6,942
10.90
CatNose
6,942
10.90
DogEye
11,580
18.19
DogFace
5,790
9.10
DogMouth
5,790
9.10
DogNose
5,790
9.10

Image size
Image dimensions in the training set ranged from 4x4 to 2219 x 2219 pixels, a total of 7,861 sizes. The median short side of images in the training set was 78 pixels. 37.2% of training set images had a short side of less than 64 pixels. 69% of training set images were square.

Distribution of image dimensions differed by class in the training set. Images in the CatFace class had median dimensions of 182 x 215 pixels. Images in the DogEye and DogNose classes had average dimensions of 62 x 62 pixels. A model that included image width and height features as inputs had a cross-validated accuracy of 45.8% on the training set, substantially better than the majority-class accuracy of 21.8%. Image dimensions in the test set (median short side of 77 pixels, 40.4% under 64x64 pixels) were similar to those in the training set.

Image quality and duplicates

The small images were less sharp than other images. Sharpness measurements were based on the variance of a Laplacian filter over 128 x 128 grayscale images. Sharpness measurements were made on a random sample of 2,000 training images. Median sharpness scores were 17.9 for images with a shorter side length of less than 48 pixels (n=431) and 379.2 for images with a shorter side length of more than 48 pixels (n=1,569). Lower scores indicate a blurrier image.

This dataset contains 30 training images. There are 15 repeated training images that form 15 pairs of training images with the same MD5 hash. The pairs of training images do not have different labels. No training image is the same as any test image. There are few repeated images. Assigning one member of a repeated pair to the training set and the other to the validation set may lead to overly optimistic validation set performance results. All sets retain intact repeated pairs of images.

Task 2: Data Preprocessing and Pipeline Preparation

Labels and train/validation split

All labels are assigned integer label values from 0 to 7 based on alphabetical order of class name to allow the class name to be determined from the integer values. A dataset of labeled images is randomly split into an 80/20 training and validation set while maintaining the same relative proportions of classes, with no duplicate or identically named images in both sets (using StratifiedGroupKFold from the scikit-learn library and file hash to group images, with a fixed random seed). Stratified splitting ensures equal representation of classes in both training and validation sets. Grouping by hash prevents duplicate images from being present in both sets. The training set contains 50,928 images, and the validation set contains 12,732 images . The greatest difference in class representation between the two sets is 0.01 percentage points. File output records the split to be used in future iterations. Tests confirmed that there are no duplicate paths or files in both sets.

Resizing and normalisation

Class names are integers 0 through 7 assigned alphabetically. This facilitates converting class predictions to class names. Image class labels were preprocessed prior to training and validation set creation, using 80% and 20% of images for training and validation, respectively. Stratification techniques were used during preprocessing to keep duplicate images together from the original data. Scikit-learn's

StratifiedGroupKFold class was used to create training and validation sets, with consistent seeds passed to the function (in this case, file hashes) to prevent different function calls from splitting the same images. The final training and validation sets included 50,928 and 12,732 images, respectively. The largest difference between the percentage of a given class in the training and validation sets is 0.01 percentage points . Data about the training and validation sets were written to a file. Measures were taken to prevent duplicate images or image paths in the training and validation sets.

validation set. Hash-based grouping ensures no image is shared between training and validation sets. The training set contains 50,928 images, and the validation set contains 12,732 images. Differences in class distribution between the training and validation sets are less than 0.01% . Documentation of how the training and validation sets were created is stored in a file to allow the code to be run multiple times with the same images. Other steps ensure that no two images with the same file path or byte values are included in both sets.

Data set	Images	Share of labelled data (%)
Training	50,928	80.0
Validation	12,732	20.0
Test (unlabelled)	29,443	not applicable

Augmentation

To expand the training set, some weak data augmentation techniques were used. Techniques include a 50% chance of randomly flipping the image horizontally, randomly rotating the image less than 10 degrees, randomly translating the image less than 5%, and randomly scaling the image between 0.9 and 1.1. Other data augmentation techniques include adjustments to image brightness, contrast, and color saturation. These data augmentation techniques are reasonable because facial features, including the mouth and nose, tend to be symmetrical. Vertical flipping and greater degrees of rotation were not applied to training images because they appeared to be upright. Similar data augmentation techniques were applied to validation and test images, but only one technique was applied to each validation and test image. Data augmentation of validation and test images was used to resize and normalize these images. Images depicting the same photo with different training, validation, and test augmentations.

Data pipeline

The Dataset class handles loading images from disk and converting them from their original color space to the RGB color space defined in the problem. Data loaders used a batch size of 64, randomly sampled and shuffled training examples with a set random number seed, and multiprocessing was not used on Windows laptops for technical reasons. Test images were loaded in the same order as images in the sample submission file, and we confirmed that all ids in the sample submission file were valid test image ids. Final pipeline batches have dimensions of (64, 3, 96, 96), a near-zero mean, and unit standard deviation after normalization.

Task 3: Model Selection and Pipeline Development
Model choice
For this experiment, we selected ResNet-18 (pretrained on ImageNet; He et al., 2016) as a transfer learning model. ResNet-18 is a residual convolutional neural network with 11.2 million parameters that can be trained on a laptop CPU (Intel Core i7-10750H, CPU only). Edges and textures learned while training ResNet-18 on ImageNet are expected to be useful for training on images of cat and dog faces. Other models with many parameters, including the Vision Transformer ViT-B/16 (Dosovitskiy et al., 2021), which we did not use in this experiment, have 86 million parameters and train much more slowly on a CPU. Images must be 224 by 224 pixels to use with this model. It probably takes more data and training iterations to train a new convolutional neural network model from scratch than to fine-tune ResNet-18.
Model Configuration
The network input size is 3 x 96 x 96 following preprocessing. The last fully connected layer is modified to have 8 outputs. Other layers in the network use ReLU activation. The output layer does not use activation to output unnormalized scores called logits, which are normalized in the softmax function used for training. Cross-entropy loss is suitable for training a single-label classifier with 8 classes. AdamW (Loshchilov & Hutter, 2019) optimization with a learning rate of 3e-4 and weight decay of 1e-4 helps prevent overwriting useful pretrained weights as the model is fine-tuned. A batch size of 32 and 5 training epochs allow the model to be trained on a laptop CPU with sufficient training data to estimate gradients accurately.

In the baseline model, only the last residual block (layer4) and fully connected layers were trained (8.4 million of 11.18 million total parameters). Untrained earlier layers in the network capture basic features like edges and other textures that could be helpful for other tasks and do not need to compute gradients to backpropagate over five training epochs. Training on the laptop CPU took about 13 minutes per epoch.
Setting
Value
Model
ResNet-18 with ImageNet pretrained weights
Input

3 x 96 x 96, ImageNet normalisation
Output layer
Linear layer, 512 inputs to 8 outputs (logits)
Trainable parts
layer4 and the output layer (8.40M of 11.18M parameters)
Loss
Cross-entropy
Optimiser
AdamW, learning rate 3e-4, weight decay 1e-4
Batch size
32
Epochs
5
Model selection
Weights from the epoch with the best validation accuracy
Hardware
Intel Core i7-10750H, CPU only

Training and prediction pipeline
The pipeline defines three functions related to the pipeline. The first function runs a single pass over a data loader in training or testing mode and returns loss and accuracy metrics. The second function trains the model for a given number of epochs and returns training and validation loss and accuracy after each epoch; it also saves weights that give the best validation accuracy. The second function accepts optional arguments for a learning rate scheduler and early stopping patience values, used in Section 4. The third function loads the saved weights and uses them to classify individual test images named in sample_submission.csv and checks the resulting output file against sample_submission.csv on factors such as column names, row numbers, and image IDs and class names in the correct order. Saves an output file for submission to Kaggle. A timing test on a few batches is performed before running the full script to predict the time to run an epoch. A debug mode enables running the full script on a small data sample.

Task 4: Training Strategy and Overfitting Management
Baseline learning curves
For the baseline results described in Section 3, the network was trained for 5 epochs and learning curves were generated. Training accuracy improved from 93.7% to 98.1% over 5 training epochs, and training loss declined each of the 5 training epochs, from 0.181 to 0.058. Validation accuracy improved from 97.9% to 98.78% over 5 validation epochs. Validation loss was lowest in the fifth validation epoch at 0.0355.

This network training session did not indicate overfitting to the training data. Signs of overfitting include higher training accuracy, lower validation accuracy, and rising validation loss while training loss continues to decrease. Note that training accuracy and loss are averaged across a training epoch with changing weights and random distortions to training images, while validation accuracy and loss are measured at the

end of each training epoch with fixed weights and undistorted validation images. In this session, validation loss increased during epochs 3 and 4 and decreased during epoch 5.

There are two problems. One is a mild underfitting problem, training loss is going down, and validation performance is highest in the last training epoch, so training for longer on this data should help. Another problem is that only the layer4 and output layers are being trained. This can limit the model's ability to learn good representations for this data. Variation in validation loss suggests that the learning rate may be too high in later training epochs.

Strategies applied

For changes to the learning curve, the problem was underfitting and noisy validation performance, not overfitting. These three related changes to the learning curve were made in one batch as a single improvement.

The first suggestion to improve performance is to use Cosine-annealing learning rate decay (Loshchilov & Hutter, 2017) with a learning rate that decays from 3e-4 to 0 over the 5 training epochs. Large learning rates during the initial training epochs may speed training, and small learning rates during later training epochs may decrease validation loss variation. The second suggestion to improve performance is to include more layers in fine-tuning. The training loss curve shows no signs of overfitting, suggesting that adding model parameters during fine-tuning may help. Pretrained later layers of the model are more specific to the original training task and more helpful during fine-tuning than earlier layers. Layers 3 and 4 of the model are added to the layers fine-tuned. The number of trainable parameters increases from 8.40M to 10.50M out of 11.18M total.

The following hyperparameters were not adjusted: weight decay of 1e-4 for AdamW, no learning-rate scheduling, data augmentation during training, batch normalization in the ResNet-18 network, pre-trained weights for the base network. The following features were excluded from the model due to underfitting and longer training times: dropout and stronger data augmentation techniques. There is insufficient evidence from a single experiment (lasting over an hour on a laptop CPU) to draw definitive conclusions about the impact of the two changes to the second network: learning-rate scheduling and additional trainable weights in later layers.

Setting	Baseline	Improvement run
Trainable parts	layer4 and output layer (8.40M)	layer3, layer4 and output layer (10.50M)
Learning rate	3e-4, constant	3e-4, cosine annealing (2.9e-5 in epoch 5)
Early stopping		

None
Patience of 2 epochs (not triggered)
Epochs, batch size, weight decay
5, 32, 1e-4
5, 32, 1e-4 (unchanged)
Results of the improvement run
The maximum evaluation accuracy for the improved model was 99.47% in epoch 5 and a validation loss of 0.0152. Evaluation accuracy and loss rose from an accuracy of 98.94% and loss of 0.0325 in epoch 1 to 99.47% and 0.0152 in epoch 5. Unlike the baseline model, validation loss did not increase for any epoch in the improved model. Training accuracy was 99.51% at the end of training for the improved model, close to maximum evaluation accuracy. There is no difference between training accuracy and evaluation accuracy, indicating no overfitting. Training was not stopped early because evaluation accuracy continued to rise through training.

Run
Best epoch
Validation accuracy
Validation loss
Training accuracy
Minutes per epoch
Baseline
5
0.9878
0.0355
0.9810
12.8
Improvement run
5
0.9947
0.0152
0.9951
14.0

The results show that all modifications helped performance. The study did not evaluate the separate effects of each change. The extra layer adds parameters and training time per epoch: 14.0 minutes per epoch compared to 12.8 minutes per epoch on a laptop CPU, about a 25% increase in parameters and a 9% increase in time. Validation loss decreased up to epoch 5; more training could provide additional benefits.
Task 5: Model Evaluation and Kaggle submission
Validation process
The best model from the improvement stage is the model with the best weights from the validation stage (12,732 images). The best model had 99.47% accuracy, 0.0152 loss, and a 0.9948 macro F1-score on 12,732 validation images. Accuracy is the main criterion used by Kaggle to rank submissions. Other metrics for which scores were calculated include class-specific precision, recall, and F1 score. Scikit-learn code (Pedregosa et al., 2011) was used to calculate metrics on an imbalanced set of classes. Recall for a class is equivalent to accuracy for the

class.

26

Class
Precision
Recall
F1-score
Images
CatEye
0.9953
0.9953
0.9953
2,777
CatFace
1.0000
0.9993
0.9996
1,388
CatMouth
0.9843
0.9906
0.9874
1,388
CatNose
0.9891
0.9820
0.9855
1,389
DogEye
0.9948
0.9965
0.9957
2,316
DogFace
0.9991
1.0000
0.9996
1,158
DogMouth
0.9974
0.9991
0.9983
1,158
DogNose
0.9991
0.9948
0.9970
1,158

This code submission was made to Kaggle competition by Kaggle user Outer Skye. Competition score on the Kaggle public leaderboard is 0.9723, an improvement of 0.61 percentage points over the 0.9662 score of a baseline submission.

Confusion Matrix
Of 12,732 validation images, 67 images were incorrectly classified. The most common misclassifications were CatNose images incorrectly classified as CatMouth and CatMouth images incorrectly classified as CatNose, 30 of 67 misclassified images. CatEye and DogEye images were incorrectly classified by species but not by feature, eight images each way. Other misclassifications between pairs of classes occurred only four times or fewer.

Kaggle submission
Predictions were generated for the 29,443 test images using the final model. The results were saved to a file called submission_task4.csv and reviewed against sample_submission.csv to ensure that the column names and number of rows matched, the test image IDs were in the same order, and the predicted class names were appropriate for submission . The distribution of classes in the test set was similar to that of the corresponding classes. Training set proportions differ from the expected training set proportions by less than 0.6 percentage points for all other classes and by 1.06 percentage points for CatNose, for which the classifier made 1.06 percentage points fewer predictions than expected given the 9.84% (10.90% in the training set) prediction rate for CatNose in the test data.

The enhanced submission was uploaded to Kaggle as Outer Skye and scored 0 .9723 on the public leaderboard, ranking 23rd when these images were captured. This was a 0.61 percentage point gain over the base submission score of 0.9662.

Validation compared with the kaggle leaderboard
The table shows how well the model performed on validation data compared with how well it performed on Kaggle test data, for both the enhanced and baseline models. Performance on test data was 2.2 percentage points lower for both models than on validation data. The gap in performance between the two models was similar for both test and validation data, 0.69 percentage points for the validation data and 0.61 percentage points for the test data. Changes to increase model capacity and the learning-rate schedule did not help close the validation-test performance gap for the enhanced model, but the improved performance of the enhanced model on the validation data generalized to improved performance on the test data.

Model
Validation accuracy
Validation loss
Kaggle public score
Difference (points)
Baseline
0.9878
0.0355

0.9662
2.16
Improvement run
0.9947
0.0152
0.9723
2.24

There are several possible reasons for differences in image size. Images in the validation set were drawn from the same labeled dataset as the training data, and some of the same or very similar images are present in both the training and validation sets. The optimal epoch for each model was selected using accuracy on the validation set for images in the validation set, the public leaderboard score is artificially high because images in the validation and public leaderboard sets are not exactly the same. The public leaderboard score is more volatile than the validation set score because it is based on fewer test images. Also, a higher proportion of very small images was seen in the test set than in the training set with labels, 8.2% of test set images were under 32 pixels, compared with 6.5% of labeled training set images, as noted in Section 1. Possible reasons for differences in image size between the training and test sets were not investigated in this post.

Task 6: Results Analysis and Improvement strategy

Interpreting the results

The table below presents results from three models. The 95% confidence interval of the validation score for the improved model is 99.33%-99.59% (Wilson score interval calculated with 12,732 images), for an estimated validation standard deviation of 0.13 points from sampling error. The improved model performs significantly better than the baseline model: 118 images correctly identified by the improved model were incorrectly identified by the baseline model, and 30 images correctly identified by the baseline model were incorrectly identified by the improved model (McNemar's exact test, p = 1.5e-13). Similar improvements were observed on Kaggle, with a gain of 0.61 points compared with 0.69 points on validation.

The validation scores are too high. The Kaggle scores were 2.16 and 2.24 points lower, more than just sampling error, than the validation set scores. They are unlikely to come from the same distribution as the validation set scores. The public Kaggle score uses some but not all of the test set data.

Model
Validation accuracy
Kaggle public score
Gap (points)
Baseline (layer4 trained, augmentation)
0.9878
0.9662
2.16
Improvement run (layer3 and layer4, augmentation)
0.9947

0.9723
2.24
Improvement run without augmentation
0.9954
0.9668
2.86
Misclassifications and difficult classes
Of 67 validation errors, 47 (70.1%) involved the correct animal species and incorrect feature, 18 (26.9%) involved the incorrect animal species and correct feature, and 2 (3%) involved both incorrect identifications. The most frequent error types were related to the CatNose and CatMouth images, with 20 and 10 one-way errors, respectively, for a total of 45% of all errors. The second most frequent errors were related to the CatEye and DogEye images, with eight one-way errors each.
The top-confidence incorrect results include cases with small, low-contrast, or dark images (8 of 12 cases had a minimum image size of less than 50 pixels), multiple objects partially or fully visible in the cropped region, and incorrectly labeled objects, including a bulldog labeled DogMouth. The mean confidence of incorrect results was 0.753, compared to 0.997 for correct predictions. The 17 errors with a confidence of 0.9 or greater were likely mislabeled.

Reasons for performance limit
Images with a shortest dimension of less than 32 pixels make up 6.6% of validation images and 26.9% of incorrect classifications. Accuracy for these images is 97.85%, compared with 99.37%-99.73% for other images. Test set images make up 8.2% of test data. To control for test set composition, adjustments were made to the validation set results. Accuracy for the adjusted validation set is 99.44%, compared with 99.47% for other images. Thus, differences in image size explain 0.03 of the 2.24-point difference in accuracy. The hypothesis in section 1 that smaller test images will be harder to classify is not confirmed.

Shortest side (px)
Images
Errors
Accuracy
Validation (%)
Test (%)
Under 32
837
18
0.9785
6.6
8.2
32 to 47
1,937
10
0.9948
15.2

15.4
48 to 63
1,915
7
0.9963
15.0
16.8
64 to 95
2,994
8
0.9973
23.5
21.9
96 to 127
1,910
12
0.9937
15.0
13.9
128 and over
3,139
12
0.9962
24.7
23.8

Other explanations were possible: similar samples in each subset of the split, with true duplicates excluded; test samples that differ in source or conditions from the labeled samples. The following information suggests the second explanation.

Effectiveness of the design choices

Preprocessing. Resizing images to 96 by 96 pixels and ImageNet preprocessing matched those used when training the pre-trained network. Other preprocessing techniques were not tested. Images smaller than 32 pixels were scaled up by three times or more with no added information and had the lowest accuracy. The impact of other image sizes was not investigated.

Augmentation. An ablation study was conducted without augmentation and with other modifications. Validation scores were 99.54% with augmentation and 99.47% without, a small but not statistically significant difference (28 out of 64 images were classified correctly only with augmentation and 36 only without; p = 0.38 by McNemar's test). Kaggle scores were 0.9668 with and 0.9723 without augmentation. Training accuracy exceeded validation accuracy by 0.41 in the experiment without augmentation and by 0.04 with augmentation. Kaggle accuracy exceeded validation accuracy by 2 .86 in the experiment without augmentation and by 2.24 with augmentation. Findings indicate that augmentation increased test data accuracy as measured on Kaggle but not validation data accuracy because validation data more closely resembled the training data. The data shown are from one run for each condition. Kaggle scores were reported only on a portion of the test set. Conclusions are speculative. Training time was 1.3

minutes longer per epoch in the experiment with augmentation, 14.0 minutes compared to 12.6 to 12.9 minutes without augmentation.

Modeling and training methods. In my experiments, I obtained a validation accuracy of 98.78% on ResNet-18 by training only layers 4 and output. This suggests that pretrained models work well on this task. I later trained layer 3 as well, using a cosine learning rate schedule, and saw a validation accuracy of 99.47%. On Kaggle, without data augmentation, I achieved a score of 0.9668, close to the baseline score of 0.9662, showing that training layer 3 and using a cosine learning rate schedule offer no benefit on Kaggle without data augmentation. Separating these effects would take three more runs of about an hour each.

Improvement strategy
Five recommendations, ordered by potential benefit. (1) Stronger validation methods to address a lack of evidence of benefits with image augmentation, including perceptual hashing and k-fold validation techniques to test differences among runs and adversarial validation methods to train a classifier to differentiate training and test images and understand differences between the sets. (2) Stronger image augmentation techniques, including those that proved successful in the Kaggle competition, such as random resized crops and blurring or repeated upscaling and downscaling to simulate lower-resolution images and greater variation in brightness. (3) Extensive fine-tuning. Validation error continued to decline across training epochs. Considerations for further fine-tuning include adding training epochs and unfreezing all layers with lower learning rates for early layers. Using larger input images and models, including deeper versions of ResNet and the Vision Transformer (Dosovitskiy et al., 2021), can help with lower-resolution images but require many hours of CPU time. (4) Ensembling techniques and mirror-image test-time augmentation reduced the number of errors from 67 to 64 and 61. Variations of 3 to 6 errors likely reflect run-to-run variation and should be confirmed in the Kaggle competition. (5) Review of the 17 high-confidence errors to check for mislabeling.

Model
Validation accuracy
Errors
Baseline
0.9878
155
Improvement run (final model)
0.9947
67
Improvement run and mirrored image
0.9952
61
Baseline and improvement run averaged
0.9950
64

Strengths and weaknesses of the final solution
Strengths: No loss of data by splitting the data, validation set accuracy of 99.47% and F1 scores of at least 0.985 across classes, Kaggle competition score of 0.9723, one-hour training time on a CPU, and use of statistical methods to validate claims in the paper.

Weaknesses: 2.2-point difference between the Kaggle competition score and other results indicates that the validation set is too simple, only one trial conducted for each configuration, other variables present that aid in classification tasks, small input size, no hyperparameter optimization, and no discussion of problems with small and mislabeled images.

Task 7: Conclusion and Reproducibility
Conclusion
The top-performing ResNet-18 classifier had a 99.47% validation accuracy score and a 0.9723 score on Kaggle. Fine-tuning two residual blocks with a cosine learning rate improved validation and Kaggle scores by about 0.69 and 0.61 points, respectively. Data augmentation, which did not improve the validation score, improved the Kaggle score by about 0.55 points. Common errors on test images included small or poorly defined images and errors in the CatNose and CatMouth classes. A gap of about 2.2 points between validation and test scores on Kaggle is a key area for future research. Increasing the validation score may reduce this gap.
Reproducibility
All code is in one notebook, Kaggle_Competition_23206450.ipynb, which can be run in order from the first cell to the last using packages in requirements.txt and competition data files in the same directory. A fixed seed of 42 is used to create consistent test and training splits and orderings and to set initial weights. The test and training splits are recorded in split.csv. It takes about 3.5 hours to run the notebook from start to finish on the CPU (Intel Core i7-10750H) used here and completes three training runs. Cells that would carry out training are not run if saved weights are available. The GitHub repository [link] includes the Kaggle notebook and competition, requirements.txt, final submission (submission_task4.csv), other submissions, and weights and training output. The Kaggle username is Outer Skye. Packages used in the notebook include those for PyTorch (Paszke et al., 2019), torchvision (TorchVision maintainers and contributors, 2016), scikit-learn (Pedregosa et al., 2011), NumPy (Harris et al., 2020), pandas (McKinney, 2010), Matplotlib (Hunter, 2007), Pillow (Pillow contributors, n.d.), and ImageNet pretrained weights (Deng et al., 2009).
Decisions made against AI suggestions
Claude's initial revision to the preprocessing code suggested an input image size of 224 by 224 pixels. Claude's second revision suggested an input image size of 128 by 128 pixels and a reduction in the number of training epochs from 5 to 3 to reduce training time. These suggestions were not adopted. The input image size was reduced to 96 by 96 pixels to reduce training time on a CPU. The number of training epochs was set to 5

to facilitate comparison between baseline and improved models. The settings used to generate the results in this paper were 96 by 96 pixels and 5 training epochs.

The approximations of claude were also wrong as it did take longer than expected due to background processes running in the background.

References

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. In 2009 IEEE Conference on Computer Vision and Pattern Recognition (pp. 248–255). IEEE. https://doi.org/10.1109/CVPR.2009.5206848

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In International Conference on Learning Representations. https://arxiv.org/abs/2010.11929

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., . . . Oliphant, T. E. (2020). Array programming with NumPy. Nature, 585(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (pp. 770–778). IEEE. https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Loshchilov, I., & Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. In International Conference on Learning Representations. https://arxiv.org/abs/1608.03983

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In International Conference on Learning Representations. https://arxiv.org/abs/1711.05101

McKinney, W. (2010). Data structures for statistical computing in Python. In Proceedings of the 9th Python in Science Conference (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., . . . Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32. https://arxiv.org/abs/1912.01703

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf

Pillow contributors. (n.d.). Pillow (PIL Fork) documentation. https://pillow.readthedocs.io

TorchVision maintainers and contributors. (2016). TorchVision: PyTorch's computer vision library [Computer software]. GitHub. https://github.com/pytorch/vision

6 Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How transferable are features in deep neural networks? Advances in Neural Information Processing Systems, 27. https://arxiv.org/abs/1411.1792